

IMBKA + Vortex UAV Path Planner

Simple code explanation for the HLS engine and PYNQ-side runner

Document purpose This guide explains what the model does, how the path is generated, and what the important lines of the HLS and Python code mean. The language is intentionally simple so the document can be used for a project explanation, viva preparation, or implementation notes.	
Item	Meaning
Target board	PYNQ-Z2 / xc7z020
Main HLS file	imbka.cpp
Header file	bka_dynamic.h / imbka_header.cpp
Python runner	imbkatwo_pynq.py
Map size used by code	500 x 500 coordinate space
Output	A list of x/y path points, iteration count, and goal reached flag

1. What The Model Does

The model is a hardware-accelerated UAV path planner. It receives a start point, a goal point, and a list of circular obstacles. It then generates a path from the start to the goal while trying to avoid obstacles and map boundaries.

The path is not drawn all at once. The algorithm moves step by step. At every step, it asks: Which direction should the UAV move next? The answer comes from two cooperating parts: the Black Winged Kite Algorithm part and the vortex obstacle-avoidance part.

Part	Role in the path
IMBKA / BKA part	Provides the main goal-seeking behavior. It creates candidate directions, scores them, updates them using attack and migration behavior, and selects the best movement direction.
Vortex part	Activates near obstacles. It decides whether to go clockwise or counter-clockwise around an obstacle, then blends that avoidance direction with the BKA direction.
Map-aware corridor scoring	Improves the vortex side decision. It checks both possible sides using all known obstacles, wall limits, and progress toward the goal.
Hardware support logic	Handles AXI interfaces, fixed-point types, buffers, HLS pragmas, and output

	writing for the FPGA.
--	-----------------------

2. How The Path Is Generated

The full path is produced by repeating one movement decision until the UAV reaches the goal or the maximum iteration count is reached.

1. Read start point, goal point, and obstacle list.
2. Clamp values so they stay inside the allowed map range.
3. Initialize four kite directions. One points toward the goal, and the others give side and reverse directions.
4. For each iteration, measure the direction from the current position to the goal.
5. Score each kite direction using goal distance, obstacle risk, and wall risk.
6. Update kite directions using IMBKA attack and migration equations.
7. Choose the best kite and turn it into a normal BKA movement step.
8. Check whether an obstacle is close enough to require avoidance.
9. If no obstacle is close, follow the BKA step.
10. If an obstacle is close, use vortex avoidance and corridor scoring to choose the better side.
11. Blend the BKA step and vortex step according to how close the obstacle is.
12. Apply velocity smoothing, maximum step limit, wall checks, and collision correction.
13. Save the new x/y position into the output path buffers.

Path decision in one sentence: best BKA direction + obstacle threat check + best vortex corridor = next UAV position

3. Simple Control Logic

The planner behaves differently depending on whether the UAV is far from obstacles or close to obstacles.

Situation	What dominates	What the UAV does
Far from obstacles	BKA	Moves mainly toward the goal using the best kite direction.
Near an obstacle but goal direction is clear	BKA with a small repulsion	Keeps moving toward the goal while pushing slightly away from the obstacle.
Near an obstacle and direct path is blocked	Vortex plus BKA	Moves around the obstacle using the better clockwise or counter-clockwise corridor.
Stuck or hardly moving	Recovery behavior	Flips the vortex side and resets kite directions to escape looping.

4. Important Variables And Constants

These constants set the planner's behavior. They are found near the top of imbka.cpp.

Lines	Name	Simple meaning
44	LOOKAHEAD_MARGIN	How close an obstacle can be before avoidance becomes important.
45	VISUAL_PADDING	Extra safety space around the obstacle.
46	MAX_STEP	Maximum distance the UAV can move in one iteration.
47	ROBOT_RADIUS	Small radius assumed for the UAV/agent.
48	MAP_LIMIT	Maximum map coordinate. The code uses a 0 to 500 map.
49	WALL_RADAR	Distance from wall where wall repulsion begins.
50	LOOKAHEAD_PX	Distance used to test a kite direction ahead of the current position.
51	MAX_INPUT_RADIUS	Maximum accepted obstacle radius.
52-54	CORRIDOR_*	Settings for the new map-aware corridor samples.
55-56	INERTIA_FREE / INERTIA_EVADE	How smoothly velocity changes in free motion vs obstacle avoidance.
57	GOAL_RADIUS_SQ	Squared goal radius. If inside this, the goal is considered reached.
58-59	BETA_STAR / SPIRAL_B	IMBKA spiral movement constants.

5. Header File Explanation

The header file defines the data types, maximum sizes, obstacle structure, and hardware function interface.

Lines	Code part	Explanation
1-2, 47	Include guard	Prevents the header from being included more than once during compilation.
6-19	Type selection	Uses normal float types during desktop C-simulation, and fixed-point ap_fixed types for HLS synthesis.
14-18	Fixed-point typedefs	Control precision and hardware cost. Smaller fixed-point widths usually use fewer FPGA resources than floating point.
21	data_t	External AXI data type. It remains float for input/output buffers and register data.
23	MAX_ITER	Maximum number of path points generated by hardware. Here it is 800.

24	MAX_OBS	Maximum number of obstacles. Here it is 20.
25	HIST_SIZE	History buffer size used to detect when the UAV is stuck.
26	N_KITES	Number of kite candidate directions. Here the planner uses 4.
28-32	Obstacle struct	Stores each obstacle as x position, y position, and radius.
34-45	bka_hw_engine prototype	Declares the top-level HLS function called by PYNQ through AXI registers and memory buffers.

6. HLS File Walkthrough

This section explains the necessary parts of imbka.cpp in the order they appear.

6.1 File Header And Compile Modes

Lines	Purpose	Simple explanation
1-23	File description	States that this is an IMBKA + Vortex path planner for PYNQ-Z2 and records the timing/resource design goal.
25	GMP macro undef	Avoids a known HLS/GMP macro conflict on some toolchains.
27	Include project header	Imports the type definitions, constants, obstacle struct, and function prototype.
29-39	Simulation vs HLS mode	When BKA_NATIVE_FLOAT_CSIM is defined, the code uses float and normal sqrt for desktop testing. Otherwise, it uses ap_fixed and hls_math for FPGA synthesis.

6.2 Small Math Helpers

Lines	Function	Explanation
64-68	square_distance()	Returns $x*x + y*y$. The code uses squared distance often because it avoids unnecessary square roots.
70-77	clamp_coord()	Keeps coordinates between 10 and 490 so the UAV does not leave the useful map area.
82-102	rng_f01()	Creates a small pseudo-random number between about 0 and 1. This supports the exploration behavior of BKA.
104-121	rng_cauchy_cheap()	Creates a cheap Cauchy-like random value. This gives the kite directions some controlled variation.
126-134	exp_approx()	Approximates exponential behavior without using a costly full math library.

136-160	sin_approx() / cos_approx()	Approximates trigonometry for the spiral behavior while keeping hardware cost lower.
165-174	sqrt_dist()	Computes square root using normal math in C-sim and hls::sqrt in hardware.
176-199	levy_cheap()	Creates a small Levy-like exploration step used in the BKA migration behavior.
201-219	normalize_pair()	Converts a vector into a unit direction so movement has direction but controlled length.

6.3 BKA Fitness Function

The function `imbka_fit()` is the main scoring function for candidate kite directions. A lower score means a better direction.

Lines	Action	Plain explanation
224-229	Function inputs	Receives one candidate direction, the current position, the goal position, and all obstacles.
231-232	HLS control	Keeps this function separate and gives it a small latency target.
233-236	Goal cost	Looks ahead in the candidate direction and measures how far that future point is from the goal.
238-251	Obstacle cost	Adds penalty if the future point is too close to any obstacle.
253-259	Wall cost	Adds penalty if the future point is too close to the map border.
261	Return score	Final score is goal cost + obstacle cost + wall cost. The best kite has the smallest score.

```
fitness = goal_cost + obstacle_cost + wall_cost
lower fitness = better direction
```

6.4 Map-Aware Corridor Cost

This is the main improvement added to stop the path from always behaving like it wants to go left. It compares the two possible ways around an obstacle.

Lines	Action	Plain explanation
264-271	Function inputs	Receives one possible tangent direction, the goal direction, current position, goal, obstacle list, and side-threat value.
273-274	HLS control	Keeps the function as a controlled hardware block.

276	Start with side threat	If more obstacles are already on that side, the cost starts higher.
278-284	Sample future points	Checks four points along that possible corridor, not just the immediate next point.
286-288	Goal progress	Adds cost if the corridor does not move well toward the goal.
290-295	Wall safety	Adds cost if the corridor gets too close to the map boundary.
297-309	Obstacle safety	Adds cost if any sampled point gets too close to any known obstacle.
312-315	Direction alignment	Adds penalty if the side direction moves backward relative to the goal.
317	Return corridor score	The lower corridor score is treated as the better path around the obstacle.

6.5 Hardware Function Interface

Lines	Purpose	Explanation
323-331	Top-level HLS function	This is the function exported as the FPGA IP. PYNQ writes inputs and reads outputs through this interface.
333-343	AXI-Lite ports	Small scalar values and pointer addresses are controlled through the AXI-Lite register map.
345-347	m_axi ports	Large arrays such as obstacles and output paths are read/written through AXI master memory access.
349-354	Resource limits	Limits function and divider instances so the design does not duplicate too much hardware.

6.6 Input Loading And Initialization

Lines	Action	Plain explanation
356-359	Clamp start and goal	Ensures start and goal coordinates stay inside the map.
361-363	Clamp obstacle count	Prevents reading more obstacles than the hardware supports.
365-371	Read obstacle buffer	Copies obstacle data from external memory into local hardware memory.
373-389	Validate obstacles	Clamps obstacle x, y, and radius values into safe ranges.
391-399	Initialize history	Fills the history buffer with the start position for later stuck detection.
401-407	Declare kite arrays	Stores x/y direction and fitness for four kites. Array partitioning

		allows quick access.
408-425	Initialize kites	Creates four starting directions: toward the goal, left/right style perpendicular directions, and opposite direction.
428-440	State variables	Stores the current position, velocity, vortex state, iteration result, and goal flag.
442-445	Fixed-point iteration ratio	Hardware-only variables used to approximate $t / \text{MAX_ITER}$ without costly division.

6.7 Main Loop And BKA Direction Choice

The main loop creates one new path point per iteration.

Lines	Action	Plain explanation
450-453	Loop setup	Runs up to MAX_ITER times. It is deliberately not pipelined to reduce LUT pressure.
455-468	Goal distance	Measures distance from current position to goal. If close enough, it writes the final goal point and stops.
470-479	Raw goal direction	Creates a unit vector pointing directly toward the goal.
481-489	BKA terms	Computes spiral, Levy, and sine-based values used by the IMBKA movement update.
491-504	Evaluate kites	Scores every kite and finds the best and worst direction.
506-507	Leader direction	The best kite becomes the leader for the next BKA update.
509-537	Attack/alert behavior	Updates each kite. The worst kite may move strongly toward or away from the leader; other kites spiral around the leader.
539-563	Migration behavior	Adds exploration and migration using Cauchy and Levy-like terms.
565-579	Select BKA step	Chooses the best updated kite, mixes it with the direct goal direction, normalizes it, and creates bka_step_x/y.

6.8 Obstacle Threat Assessment

Lines	Action	Plain explanation
581-588	Threat variables	Stores nearest obstacle distance, obstacle index, left/right side threat, and obstacle direction.
593-615	Find closest obstacle	Loops through all obstacles to find the nearest surface distance.
597-601	Distance to each obstacle	Computes distance from UAV to obstacle center, then subtracts the

		obstacle radius.
603-606	Store closest obstacle	If this obstacle is closer than the previous closest one, it becomes the active threat.
607-614	Side threat	Uses a cross-product sign to estimate whether obstacles are more on the left or right of the current travel direction.
617-619	Default desired step	Before avoidance, the desired movement is simply the BKA step.

6.9 Vortex Evasion Logic

This block only matters when an obstacle is close. It prevents collisions and decides how to go around the obstacle.

Lines	Action	Plain explanation
624	Obstacle trigger	Avoidance starts only if a closest obstacle exists and its surface is within LOOKAHEAD_MARGIN.
626-632	Repulsion direction	Builds a direction pointing away from the obstacle center.
634-638	Clear path case	If moving toward the goal is also moving away from the obstacle, the code keeps BKA and adds a small push outward.
639-647	Vortex mode	If the direct route is blocked, the planner enters or continues vortex avoidance.
649-658	Compare both sides	Computes clockwise and counter-clockwise directions, then calls corridor_cost() for each.
659-668	Fresh decision	When first entering vortex mode, it chooses the side with lower corridor cost.
669-681	Periodic re-vote	While already evading, it only changes side if the other side is clearly better. This prevents direction chatter.
684-701	Frustration recovery	If the UAV has barely moved for many steps, it flips the vortex side and resets kite directions to escape a loop.
703-711	Blend weights	Computes how much vortex influence is needed. Closer obstacle means higher vortex weight.
713-717	Push-out force	Adds extra outward force if the UAV is very near the obstacle surface.
719-722	Final desired movement	Combines BKA step and vortex step into des_x/des_y.
724-726	Exit vortex state	If the obstacle is no longer threatening, vortex state is cleared.

```
des_x = bka_step_x * wb + vortex_step_x * wv
```



```
des_y = bka_step_y * wb + vortex_step_y * wv
```

```
wb = BKA weight
```

```
wv = vortex avoidance weight
```

6.10 Movement, Safety, And Output

Lines	Action	Plain explanation
731-738	Wall repulsion	Pushes the UAV away from map edges if it gets too close.
740-745	Choose inertia	Uses lower inertia while evading so the UAV can turn faster around obstacles.
747-748	Smooth velocity	Blends old velocity with the new desired movement, making the path smoother.
750-757	Maximum step limit	Prevents the UAV from moving more than MAX_STEP in one iteration.
759-760	Update position	Adds velocity to the current x/y position.
762-784	Collision correction	If the new position falls inside an obstacle, it pushes the UAV back to the obstacle boundary.
786-787	Clamp map position	Keeps the UAV inside the allowed coordinate range.
789-791	Update history	Stores the current position for stuck detection.
793-794	Save path point	Stores the current x/y into local output arrays.
796-801	Hardware time ratio update	Updates fixed-point iteration ratio for the next loop.
807-815	Write output arrays	Copies local path arrays back to external memory.
818-819	Write scalar results	Returns the number of iterations used and whether the goal was reached.

7. What Part Is BKA And What Part Is Vortex

In source-code size, the approximate split is not exact, but this is a useful way to understand the model:

Part	Approximate share	Why
IMBKA / BKA	55 to 65 percent	Candidate kites, fitness, random exploration, attack, migration, and leader selection are the core of the algorithm.

Vortex avoidance	25 to 35 percent	Threat assessment, vortex side selection, corridor cost, repulsion, and BKA-vortex blending happen near obstacles.
Hardware/interface support	10 to 15 percent	AXI ports, buffers, fixed-point types, pragmas, and output writing are required for FPGA use.

Important interpretation

Even though the vortex code is smaller than the BKA code, it can temporarily dominate the movement when an obstacle is very close. When no obstacle is close, the BKA direction is the main decision maker.

8. PYNQ Python Code Walkthrough

The Python file is the board-side control program. It loads the bitstream, allocates shared memory, writes input registers, starts the hardware, waits for completion, reads results, and plots the path.

Lines	Code part	Explanation
1-5	Imports	Loads PYNQ overlay/memory tools, plotting, numpy, struct conversion, and timing.
8-10	Constants	Matches HLS limits: 20 obstacles, 800 path points, 500 map limit.
12-22	Register offsets	These addresses match the HWH register map for start, goal, obstacle pointer, path pointers, iterations, and goal flag.
25-26	<code>_float_to_u32()</code>	Converts a Python float into the exact 32-bit pattern expected by the AXI-Lite float register.
29-30	<code>_buffer_address()</code>	Gets the physical/device address of a PYNQ allocated buffer.
33-36	<code>_write_u64()</code>	Writes a 64-bit buffer address into two 32-bit AXI-Lite registers.
39-54	<code>IMBKATwoRunner.__init__()</code>	Loads the bitstream, finds the BKA IP, allocates obstacle and path buffers, and sets default start/goal.
56-61	<code>close()</code>	Releases allocated buffers when finished.
63-76	Obstacle editing	Clears, adds, or removes obstacles. Inputs are clipped into safe ranges.
78-93	run setup	Updates start/goal/obstacles, fills the obstacle buffer, and flushes it to memory.
95-102	Write registers	Sends start, goal, obstacle pointer, obstacle count, and output buffer addresses to the IP.
104-108	Start and wait	Starts FPGA execution and waits until <code>AP_DONE</code> is set. Timeout prevents an infinite wait.

110-123	Read results	Invalidates path buffers, reads iteration count and goal flag, then builds the path array and metrics.
125-133	Return result	Stores path, time, iteration count, goal status, length, and final error in a dictionary.
135-163	plot()	Draws the FPGA path, start, goal, and obstacles using matplotlib.
166-228	IMBKAInteractiveMap	Adds mouse and keyboard controls for adding obstacles, setting start/goal, clearing, undoing, and running.
231-238	Example main block	Runs a sample map and displays the result when the file is executed directly.

9. How To Explain The Algorithm In A Viva

A simple explanation you can give:

Short explanation

The FPGA receives the start point, goal point, and obstacle map. The BKA part produces candidate movement directions and chooses the direction that best moves toward the goal while avoiding obvious risk. If the UAV gets close to an obstacle, the vortex part temporarily takes more control. It checks both possible ways around the obstacle and selects the corridor with lower risk and better goal progress. The final movement is a blend of the BKA direction and the vortex avoidance direction.

If someone asks why it is not simply going left, the answer is:

- The code does not hard-code left movement.
- It computes clockwise and counter-clockwise tangent directions around the obstacle.
- It scores both directions using all known obstacles, wall limits, and distance to goal.
- The lower-cost side becomes the chosen vortex direction.

10. Testing Summary

The HLS C-simulation version was checked using three obstacle maps.

Test case	Result	Iterations	Path length	Collisions
7-obstacle static map	Goal reached	106	590.349 px	0
Clustered failure-style map	Goal reached	98	553.647 px	0
Blocked-left/right-corridor map	Goal reached	113	610.965 px	0

Synthesis note

The code was written to stay resource-conscious by using sequential loops, fixed-point arithmetic in HLS

mode, limited function instances, and no large parallel route search. However, a new Vivado/Vitis HLS synthesis run is still required to confirm the final LUT count after rebuilding the bitstream.

11. Final High-Level Summary

The path is decided by combining global goal-seeking behavior and local obstacle-avoidance behavior.

- BKA gives the main direction toward the goal.
- The fitness function discourages directions that approach obstacles or walls.
- The vortex logic activates only when an obstacle is close.
- The new corridor scoring prevents the path from blindly choosing the same side every time.
- The final output is a sequence of x/y coordinates that can be plotted in Jupyter on the PYNQ board.

Appendix A. Very Short Glossary

Term	Simple meaning
Kite	One candidate movement direction used by the BKA model.
Leader	The best current kite direction.
Fitness	Score used to decide whether a direction is good or bad. Lower is better.
Vortex	Obstacle avoidance behavior that moves around an obstacle using a tangent direction.
Repulsion	A push away from obstacle center or wall.
Inertia	Smoothing that prevents sudden direction changes.
AXI-Lite	Register interface used by Python to send small control values to FPGA IP.
m_axi	Memory interface used by FPGA IP to read/write large arrays.
HLS pragma	Instruction to Vitis HLS about how to build hardware from C++ code.